

UNIVERSITY OF RWANDA

College of Science and Technology | School of ICT

Computer Engineering Department

Mobile Application System and Design

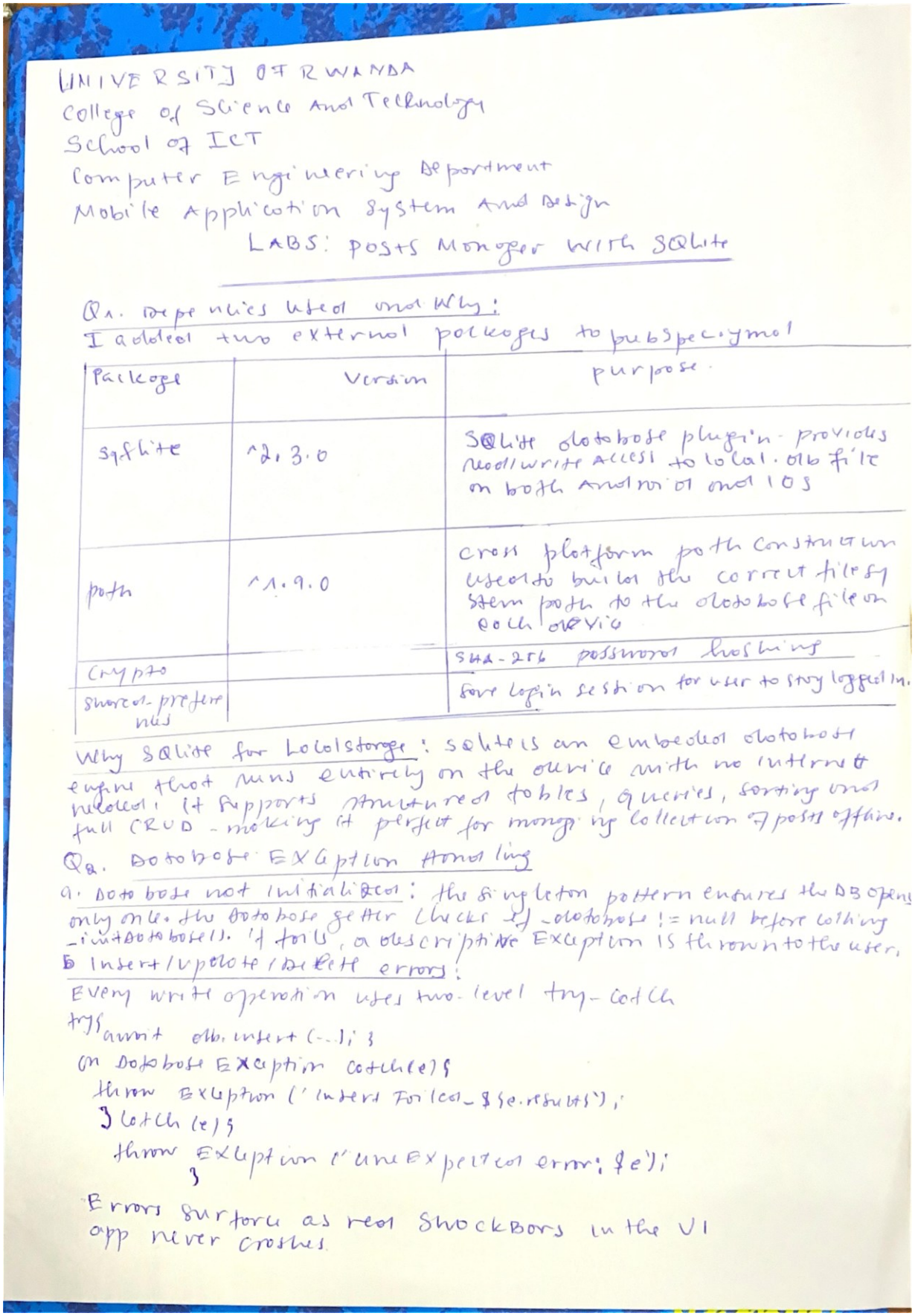
Flutter Lab 5

Offline Posts Manager with SQLite

Course	Mobile Application Development
Assignment	Lab 5 — Individual
Topic	Local Database Integration with SQLite
App Name	Offline Posts Manager
Deadline	Nyarugenge: 25 March 2026

Done By Patrick Straton Mbabazi

1. Handwritten Discussions



UNIVERSITY OF RWANDA
College of Science And Technology
School of ICT
Computer Engineering Department
Mobile Application System And Design
LABS: posts Manager with SQLite

Q1. Dependencies used and Why:
I added two external packages to pubspec.ymal

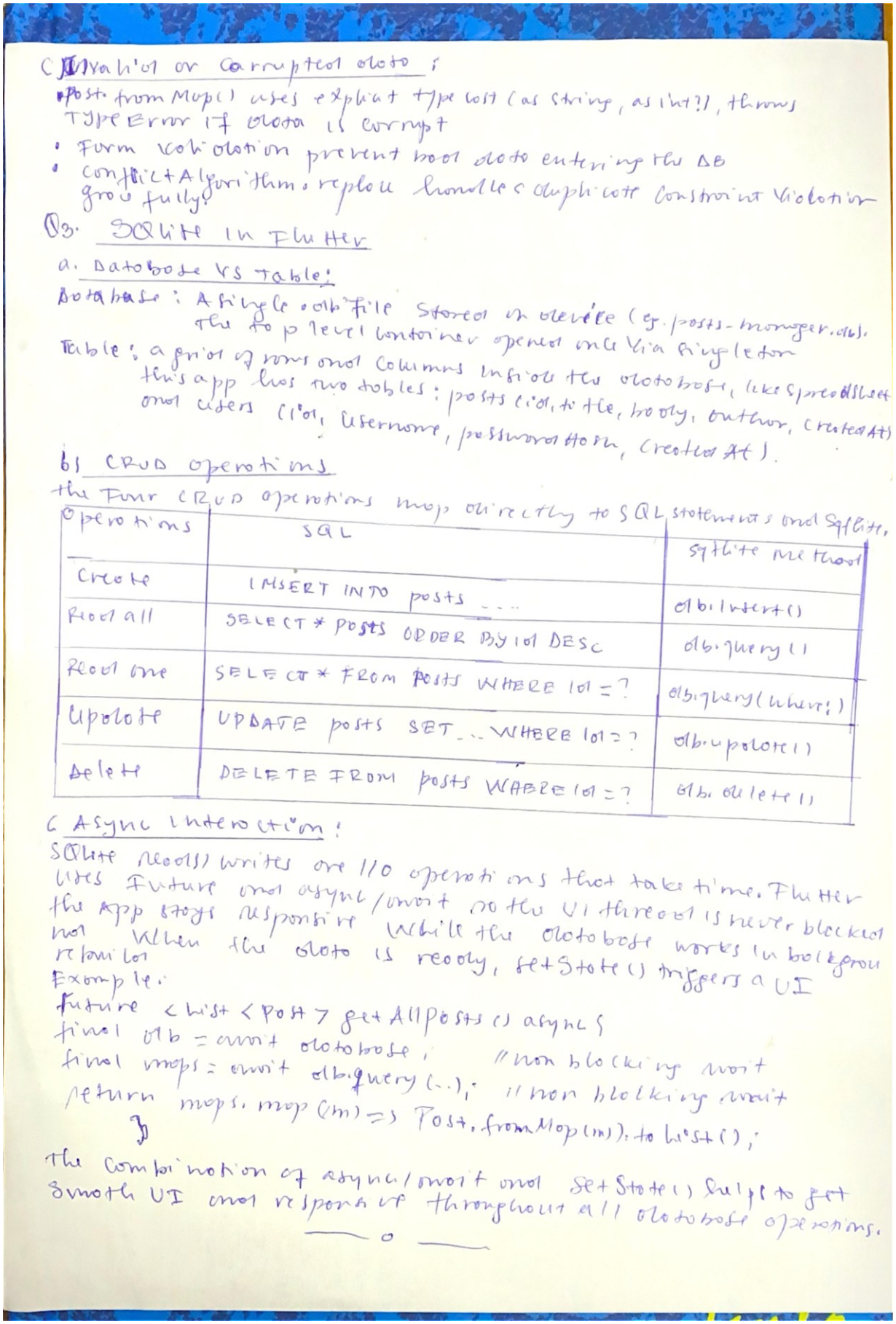
Package	Version	purpose
sqflite	^2.3.0	SQLite database plugin - provides read/write access to local .db file on both Android or and iOS
path	^1.9.0	cross platform path construction used to build the correct file system path to the database file on each device
Crypto		SHA-256 password hashing
shared_preferences		save login session for user to stay logged in.

Why SQLite for Localstorage: sqlite is an embedded database engine that runs entirely on the device with no internet needed. It supports structured tables, queries, sorting and full CRUD - making it perfect for managing collection of posts offline.

Q2. Database EXCeption Handling
a. Database not initialized: the singleton pattern ensures the DB opens only once. the database getter checks if _database != null before calling _initDatabase(). if fails, a descriptive Exception is thrown to the user.
b. Insert/update/delete errors:
Every write operation uses two-level try-catch

```
try{
  await db.insert(...);
} on DatabaseException catch(e){
  throw Exception('Insert Failed: ${e.results}');
} catch(e){
  throw Exception('Unexpected error: $e');
}
```

Errors surfaced as red SnackBars in the UI
app never crashes



C Validation or Corrupted data:
- Post.fromMap() uses explicit type casts (as String, as int?), throws TypeError if data is corrupt
- Form validation prevent bad data entering the DB
- ConflictAlgorithm.replace handles duplicate constraint violations gracefully

Q3. SQLite in Flutter

a. Database VS Table:

Database: A single .db file stored on device (eg. posts_manager.db). The top level container opened once via singleton

Table: a grid of rows and columns inside the database, like Spreadsheet this app has two tables: posts (id, title, body, author, createdAt) and users (id, username, passwordHash, createdAt).

b) CRUD operations

the Four CRUD operations map directly to SQL statements and Sqflite,

Operations	SQL	Sqflite method
Create	INSERT INTO posts ...	db.insert()
Read all	SELECT * posts ORDER BY id DESC	db.query()
Read one	SELECT * FROM Posts WHERE id = ?	db.query(where:)
Update	UPDATE posts SET ... WHERE id = ?	db.update()
Delete	DELETE FROM posts WHERE id = ?	db.delete()

C Async Interaction:

SQLite reads/writes are I/O operations that take time. Flutter uses Future and async/await so the UI thread is never blocked the App stays responsive while the database works in background not When the data is ready, setState() triggers a UI rebuild

Example:

```
Future<List<Post>> getAllPosts() async {
final db = await database;
final maps = await db.query(...); // non blocking await
return maps.map((m) => Post.fromMap(m)).toList();
}
```

The combination of async/await and setState() helps to get Smooth UI and responsive throughout all database operations.

2. App Screenshots

All 11 screenshots below demonstrate the complete functionality of the Offline Posts Manager application.

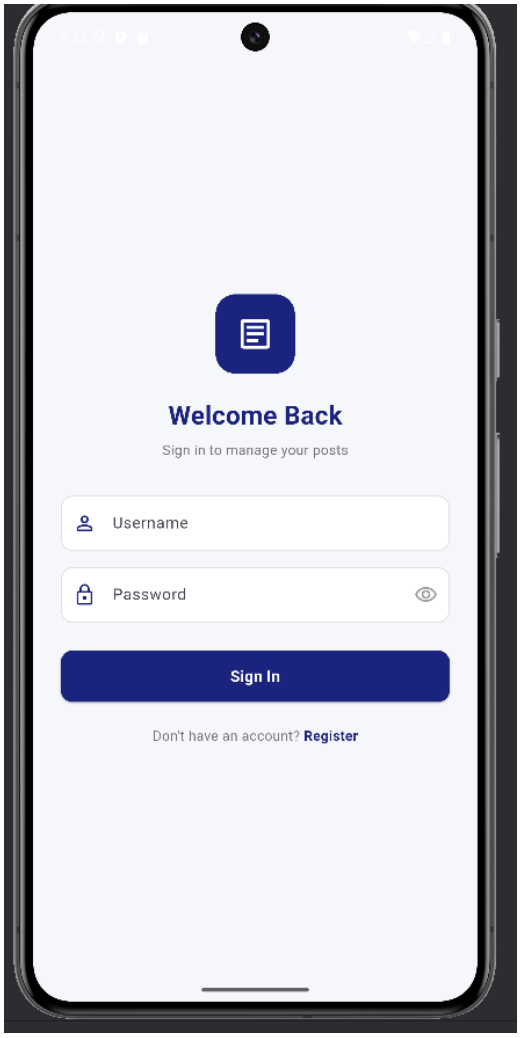


Fig 1. Login screen

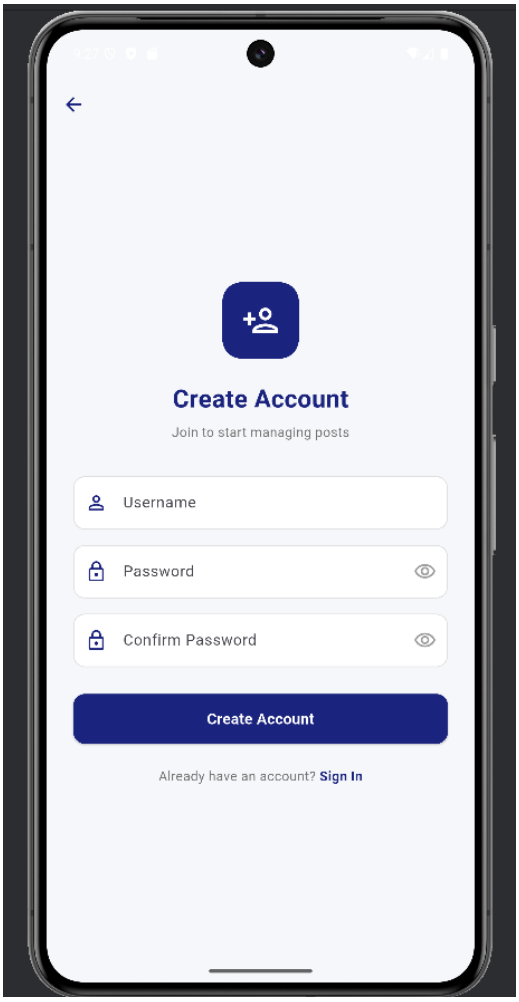


Fig 2. Register screen

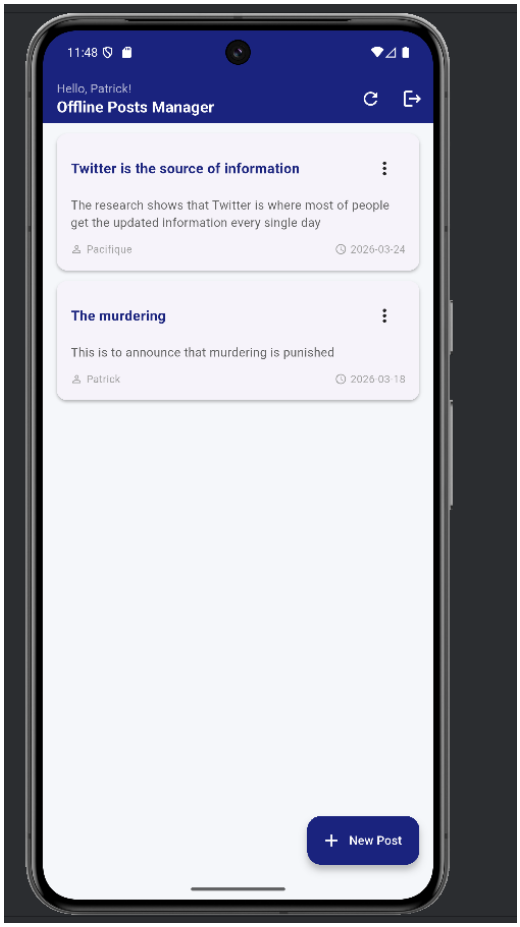


Fig 3. Posts dashboard with greeting



Fig 4. View post details

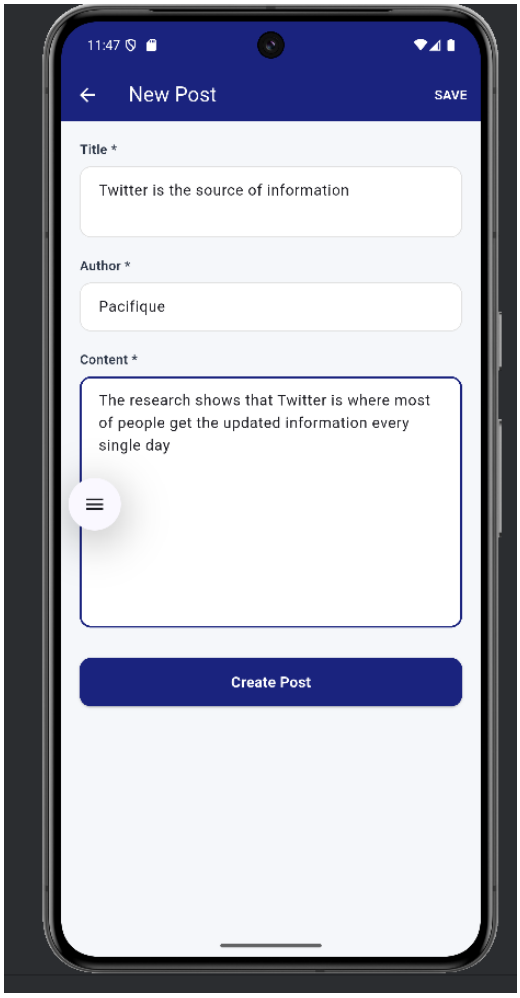


Fig 5. Add new post form

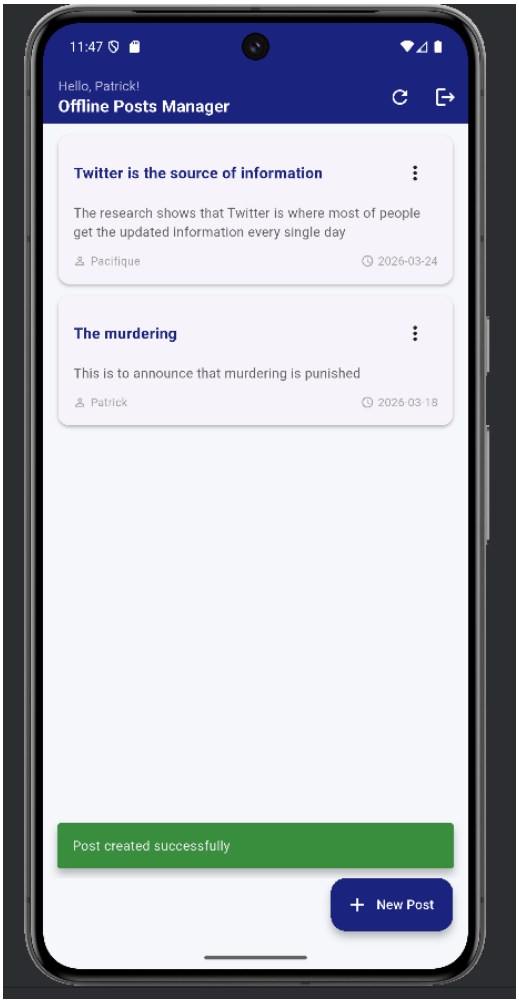


Fig 6. Post successfully created

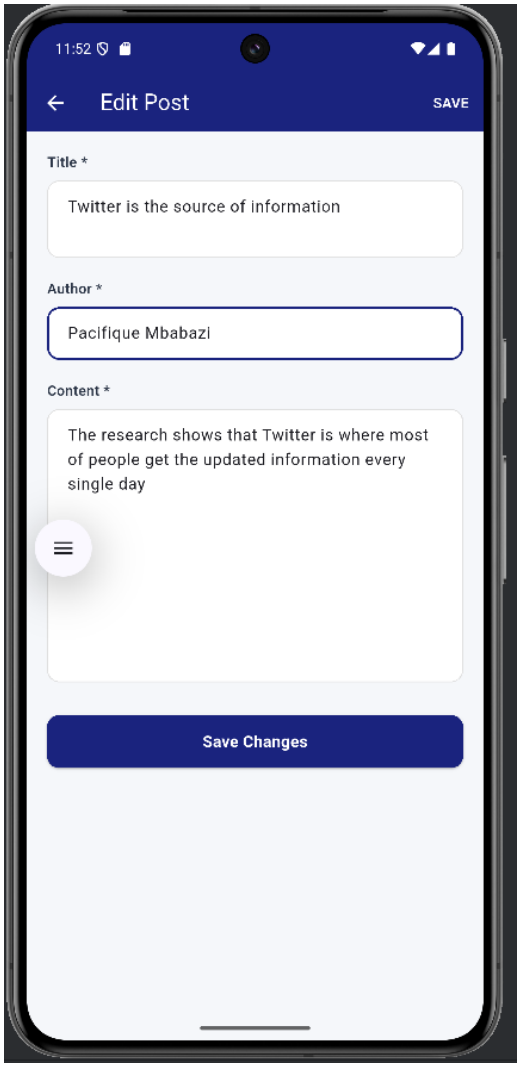


Fig 7. Edit existing post

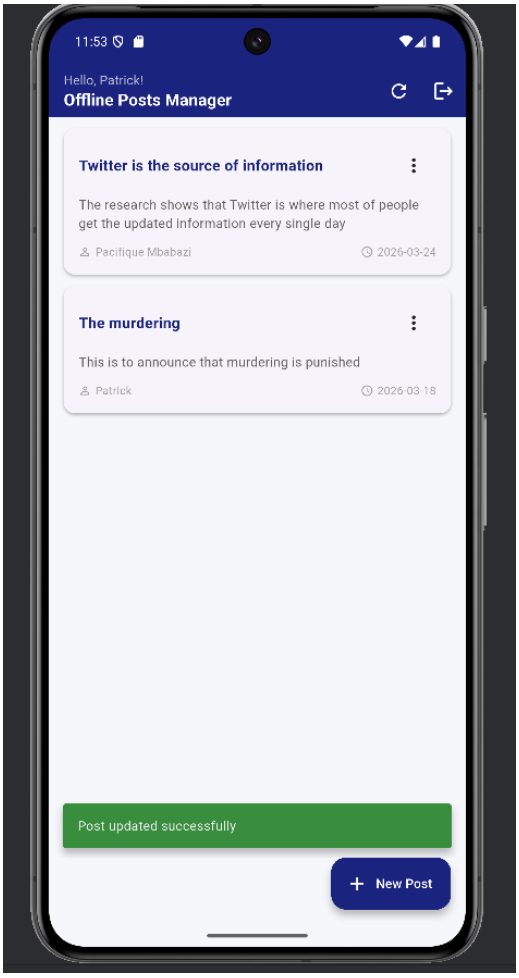


Fig 8. Post successfully edited

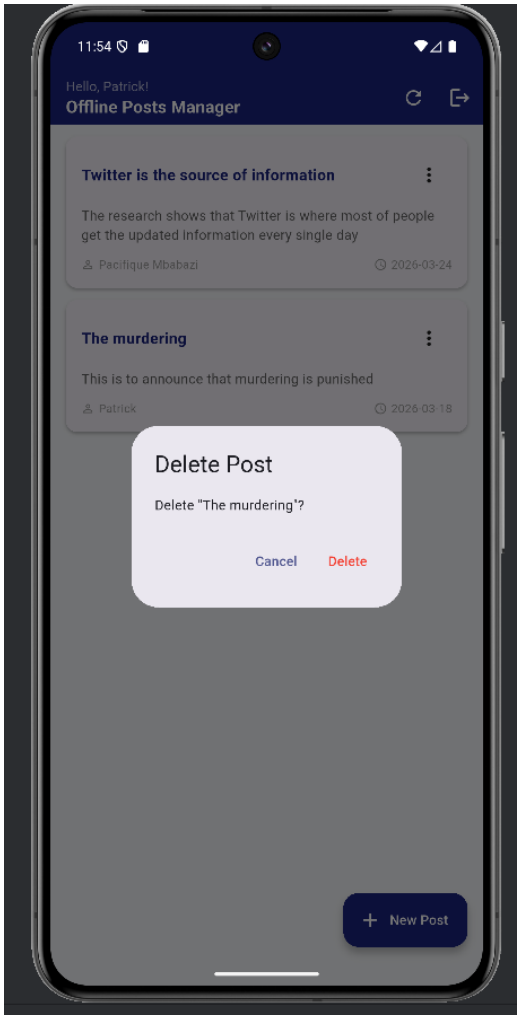


Fig 9. Delete confirmation dialog

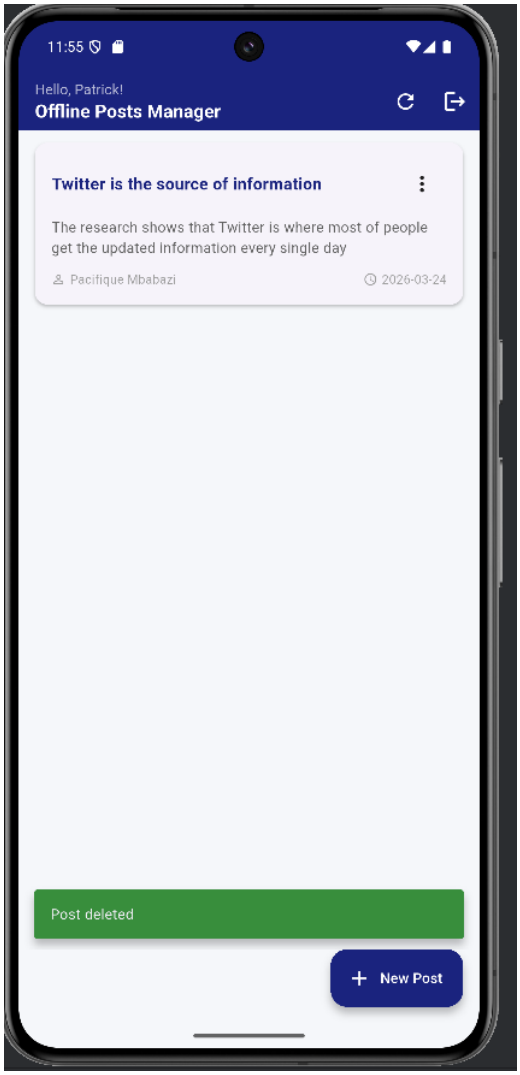


Fig 10. Post successfully deleted



Fig 11. Sign out confirmation

6. Conclusion

This lab successfully demonstrates SQLite as a local storage solution in Flutter. The Offline Posts Manager implements all five required CRUD features plus user authentication (register, login, logout) entirely offline. The implementation covers robust exception handling at three levels, asynchronous database interaction via async/await and Futures, and a clean layered architecture separating models, database, services, and UI screens.

References

- sqflite package — https://pub.dev/packages/sqflite
- path package — https://pub.dev/packages/path
- crypto package — https://pub.dev/packages/crypto
- shared_preferences — https://pub.dev/packages/shared_preferences
- SQLite Official Documentation — https://www.sqlite.org/docs.html
- Dart Async/Await — https://dart.dev/codelabs/async-await